



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

مشروع المترجمات

إعداد

عبدالله تيسير حميد

كوثر حميد الاحمد

محمد جودت محمود المعدني

معتصم وليد قصاص

بيان محمد الحلبي

إشراف

د. باسم قصيبة

المحتويات

3	أولاً: تعريف بيئة المترجم (Python, Flask & Jinja2)
4	ثانياً: بناء قواعد اللغة اللفظية والنحوية (Lexer & Parser)
5	ثالثاً: توليد الشجرة AST
6	رابعاً: توليد جدول الرموز SymbolTable
7	خامساً: معالجة الأخطاء الدلالية (Error Handling)
7	أ. الأخطاء الدلالية في لغة Python (PythonSemanticAnalyzer)
7	ب. الأخطاء الدلالية في قوالب التصميم (JinjaSemanticAnalyzer)
8	سادساً: الآلية المستخدمة لإعادة توليد المشروع عند الإضافة والحذف
9	سابعاً: استخدام كل ما سبق في البرنامج النهائي

أولاً: تعريف بيئة المترجم (Python, Flask & Jinja2)

في هذا المشروع، قمنا ببناء مترجم (Compiler) متكامل يسمح بكتابة تطبيقات الويب باستخدام لغة Python لتحديد المنطق والبيانات (Back-end)، ولغة القوالب Jinja2 مع HTML لتحديد الواجهات (Front-end). المترجم يقوم بقراءة هذه الملفات، تحليلها، وتقييمها، لإنتاج صفحات ويب ثابتة (HTML/CSS) يمكن عرضها مباشرة في المتصفح.

الهدف الأساسي من المترجم هو أتمتة عملية ربط البيانات المستخرجة من الكود المصدري (Python) بالقوالب (Jinja2)، وتوليد الكود النهائي دون الحاجة إلى تشغيل خادم ويب فعلي في بيئة الإنتاج. لتحقيق ذلك، قمنا ببناء المترجم عبر عدة مراحل أساسية تشمل: التحليل اللفظي والنحوي، بناء شجرات القواعد المجردة، التحليل الدلالي، وتقييم السياق لتوليد الواجهات الديناميكية.

من أهم الميزات التي يدعمها المترجم:

- تعريف المتغيرات والعمليات الحسابية والمنطقية في Python.
 - تقييم الشروط (if/else) وحلقات التكرار (for/while).
 - تعريف التوابع (functions) واستدعائها.
 - تعريف القوالب الديناميكية وتمرير المتغيرات إليها عبر `render_template`.
 - بناء خادم تطوير (DevServer) يسمح بإعادة توليد الصفحات برمجياً عند إضافة أو حذف البيانات.
-

ثانياً: بناء قواعد اللغة اللفظية والنحوية (Lexer & Parser)

في البداية، قمنا بكتابة قواعد اللغة اللفظية والنحوية باستخدام أداة **ANTLR4**، حيث قمنا بإنشاء ملفات قواعد مخصصة لكل بيئة. تم فصل القواعد إلى ملفات مستقلة لضمان دقة التحليل:

1. **قواعد Python:** عبر ملف `PythonLexer.g4` و `PythonParser.g4`.
2. **قواعد Jinja2 و HTML:** عبر ملف `HtmlJinjaLexer.g4` و `HtmlJinjaParser.g4`.
3. **قواعد التنسيق CSS:** عبر ملف `cssLexer.g4` و `cssParser.g4`.

حيث تم تجميع رموز لغة `Python (Tokens)` وكتابتها في الملف الخاص بال-Lexer

ثم تم استخدام هذه الرموز لبناء قواعد المحلل النحوي (Parser) الذي يفهم بنية الكود (مثل بنية تعريف الدالة، أو بنية حلقة `for`). وبعد ذلك قمنا باستخدام أداة **ANTLR** لتوليد ال-Lexer وال-Parser المحلل اللفظي والقواعدي بناءً على هذه القواعد.

ثالثاً: توليد الشجرة AST

الكلاسات التي تم توليدها باستخدام ANTLR تعطينا Parse Tree، ولكننا نحتاج لتحويلها إلى AST (Abstract Syntax Tree) لنتمكن من استخدامها بمرونة في المراحل التالية من الكومبايلر (التحليل الدلالي وتقييم الكود).

قمنا بإنشاء حزم (Packages) مخصصة مثل `python.ast` و `ast.jinja`. تعتمد هذه الشجرة على كلاسات ترث من الصف الأساسي `ASTNode` للـ Python و `Node` للـ Jinja. وبما أن الـ AST تحتوي على القواعد المنطقية للغة، فقد قمنا بإنشاء صف (Class) خاص بكل قاعدة.

مثلاً، عقدة الـ `IfNode` تحتوي على الشرط (`condition`)، وجسم الشرط (`thenBlock`)، وجسم العكس (`elseBlock`). وبذلك، تحوي الـ AST على كلاسات تجسد مفاهيم البرمجة الكائنية كـ `SOOP` والـ `Polymorphism` والـ `Abstraction`، حيث يتم تخزين مجموعة من القواعد المختلفة في حاوية واحدة (مثل قائمة من `StatementNode`).

وكذلك قمنا بإنشاء كلاسات `Visitor` ترث من الصفوف المولدة من ANTLR (مثل `PythonParserBaseVisitor` و `HtmlJinjaParserBaseVisitor`) تقوم هذه الكلاسات بزيارة عقد شجرة التحليل (Parse Tree) وتحويلها إلى العقد المخصصة في شجرتنا AST.

ملاحظة أكاديمية: تم تجهيز جميع العقد بمتغير يحفظ رقم السطر (`lineNumber`)، ليتم استخدامه لاحقاً في الإبلاغ عن الأخطاء بشكل دقيق للمستخدم، كما تم توفير التابع `printTree` لطباعة الشجرة بشكل شجري مقروء.

رابعاً: توليد جدول الرموز SymbolTable

سوف نعتبر جدول الرموز هو عبارة عن هيكل بيانات SymbolTable يحتوي على قائمة من النطاقات (Scopes). كل نطاق (Scope) مثل GlobalScope أو LocalScope هو كائن مؤلف من قائمة من الرموز Symbols التي تم تعريفها داخله. وكل نطاق داخلي (مثل نطاق دالة) يمتلك مؤشراً على النطاق الأب (Enclosing Scope).

يحتوي SymbolTable على توابع لتعريف الرموز (define) والبحث عنها (resolve) صعوداً من النطاق الحالي حتى النطاق العام. في بداية عمل المترجم، يتم تهيئة GlobalScope بجميع الأنواع الأساسية (مثل int, float, string) وكذلك الدوال المدمجة في البايثون (مثل len, range, str) ليتم اعتبارها رموزاً صحيحة أثناء التحليل.

وتم في دوال الـ Semantic Analyzer، عند المرور بتعريف أي متغير (AssignmentNode) أو دالة (FunctionDefNode)، يتم إضافته كرمز جديد إلى كائن SymbolTable الخاص بالبرنامج، مما يمنع الأخطاء المستقبلية ويسهل عملية تتبع دورة حياة المتغيرات.

خامساً: معالجة الأخطاء الدلالية (Error Handling)

وهو من أهم الأقسام في المترجم، حيث احتجنا إلى كلاسين منفصلين (بالإضافة إلى جدول الرموز) لتحقيق معالجة الأخطاء بشكل كامل وفصل أخطاء البايثون عن أخطاء قوالب التصميم. الهدف من هذا القسم هو اصطيايد الأخطاء التي لا يستطيع الـ Parser اكتشافها لأنها صحيحة نحوياً لكنها خاطئة منطقياً (دلالية). وهذه هي الفائدة الأكبر من التحقيق الدلالي، حيث توفر مرونة عالية للغة وسهولة في التقاط الحالات الخاصة (Edge Cases).

وسنورد هنا حالات الأخطاء الدلالية التي تم معالجتها بشكل دقيق في المترجم:

أ. الأخطاء الدلالية في لغة: Python (PythonSemanticAnalyzer)

1. **إعادة تعريف دالة في نفس النطاق:** عند مرور المحلل بعقدة `FunctionDefNode`، يتحقق مما إذا كان اسم الدالة موجوداً مسبقاً في النطاق الحالي لجدول الرموز. وإذا كان الأمر كذلك، يتم رمي خطأ `Redeclaration of function in the same scope.`

2. **إعادة تعريف صنف (Class):** وبالمثل عند عقدة `ClassDefNode`، يتم رمي خطأ `Redeclaration of class.`

3. **استخدام متغير غير معرف في الإسناد التراكمي (Augmented Assignment):** عند استخدام `=` أو `+=` في عقدة `AugmentedAssignNode`، يجب أن يكون المتغير معرفاً مسبقاً. إذا لم يكن موجوداً، يظهر الخطأ `Undefined variable used in augmented assignment.`

4. **عدم تطابق الأنواع في العمليات الحسابية (Type Mismatch):** عند التحقق من `BinaryOpNode` للعملية `+`، يكتشف المحلل محاولة دمج نص (string) مع رقم (int) أو `float` ويرمي خطأ `Type mismatch: cannot add string and int.`

5. **استدعاء تابع غير معرف:** في عقدة `FunctionCallNode`، يتحقق المحلل من وجود اسم التابع في جدول الرموز. وإذا لم يُوجد يتم رمي خطأ `Call to undefined function.`

6. **ورود `return` خارج جسم تابع:** في عقدة `ReturnNode`، يتحقق المحلل من عداد عمق الدوال (`functionDepth`) إذا كان يساوي صفراً، يرمي الخطأ `'return' outside function.`

7. **استخدام متغير غير معرف:** عند مرور المحلل بأي `IdentifierNode` كجزء من عملية تقييم أو استدعاء، يتحقق من تواجد المتغير؛ وإلا يرمي الخطأ `Undefined variable.`

ب. الأخطاء الدلالية في قوالب التصميم: (JinjaSemanticAnalyzer)

8. **استخدام متغير غير معرف داخل الأقواس: `{{ }}`** يحلل المترجم عقدة `JinjaExpression` ويستخرج المتغيرات الحرة (`freeVariables`). ثم يتأكد من وجودها إما في الـ `SymbolTable` أو كمتغير مُمرر من البايثون (`contextVars`) أو كمتغير مدمج. وإلا يرمي الخطأ `Undefined variable in {{ expr }}`.

9. **استخدام متغير غير معرف في جمل الإسناد: `% set %`** وبـنفس الآلية، يتم فحص المتغيرات المستخدمة في قيمة الإسناد لضمان صحة تعريفها مسبقاً.

سادساً: الآلية المستخدمة لإعادة توليد المشروع عند الإضافة والحذف

نظراً لأن المشروع يعمل كمترجم يولد كود HTML ثابت (Static Site Generation) ولا يتصرف كخادم ويب تقليدي ينفذ كود البايثون مع كل طلب) مثل تطبيقات Flask الافتراضية(، فقد واجهنا تحدياً: كيف يمكن إعادة بناء الواجهات تلقائياً لتعكس إضافة منتج جديد أو حذفه من النظام؟

لحل هذه المعضلة بطريقة أكاديمية ومبتكرة، قمنا ببناء هيكلية التوليد الذكي (Dynamic Regeneration) المكونة من طرفين DevServer و PythonContextEvaluator.

1. تقييم البيانات: (Context Evaluation)

عند تشغيل المترجم، يقوم PythonContextEvaluator بتقييم (Evaluate) كود البايثون وبناء حالة الذاكرة (Memory State). حيث يقوم بالبحث عن التوابع الموجهة بالـ Decorators مثل @app.route، ويستخرج المتغيرات التي يمررها التابع عبر استدعاء render_template. يتم تغليف هذه البيانات كـ RenderCall لتمريرها إلى القوالب.

2. النقاط وتوجيه الأحداث عبر DevServer:

عند قيام المستخدم بإضافة منتج عبر نموذج واجهة المستخدم (Form POST)، يلتقط خادم التطوير DevServer هذا الطلب. بدلاً من تشغيل تطبيق Flask، يقوم الـ DevServer بما يلي:

- **حقن الطلب:** يمرر الحدث إلى PythonContextEvaluator ويطلب منه تنفيذ تابع المسار (Route) الهدف (مثلاً add_product) مع حقن بيانات النموذج (request.form).
- **تحديث حالة الذاكرة:** يقوم المقيّم (Evaluator) بتنفيذ الكود (مثلاً products.append(new_product)). وبذلك يتم إضافة العقدة الجديدة إلى مصفوفة المنتجات داخل الذاكرة الوهمية للمترجم.

3. حفظ الحالة وإعادة التوليد: (Persist and Regenerate)

- **كتابة الملف: (Persistence)** يقوم الـ DevServer بأخذ لقطة (snapshotModule) لحالة المصفوفة المحدثة، ثم يعيد صياغتها ككود بايثون ويكتبها فيزيائياً في ملف data.py. هذا يضمن بقاء البيانات حتى لو تم إيقاف تشغيل المترجم.
- **إعادة التوليد: (Compilation Cycle)** بمجرد كتابة الملف، يتم استدعاء المحرك الأساسي للمترجم Main.compile(). يقوم المحرك بقراءة الكود مجدداً، تقييم المصفوفة التي تحتوي الآن على العنصر الجديد، ويمررها كـ Context إلى JinjaRenderer.
- **هندسة الـ ForStatement:** في هذه المرحلة الأخيرة، تقرأ عقدة ForStatement في شجرة Jinja المصفوفة المحدثة. وتبدأ بالدوران (Loop) على عناصر المصفوفة لضخ وتوليد عقد HTML مثل <div> و بحيث يتم توليد بطاقة (Card) إضافية للمنتج المضاف، أو إزالة بطاقة المنتج المحذوف. ويتم تصدير ملف index.html الجديد بالبيانات المحدثة.

سابعاً: استخدام كل ما سبق في البرنامج النهائي

في النهاية قمنا بتعريف كلاس Main وتابع main حيث يتم فيه تحديد مسار ملفات البرنامج المراد عمل compile لها. ومن أجل كل ملف من الملفات المحددة يتم قراءة الملف وتمريضه في الـ Lexer والـ Parser مع استخدام الـ SymbolTable والمحللات الدلالية بالشكل الصحيح. تسلسل العملية في البرنامج النهائي:

1. **قراءة الملفات:** قراءة كافة ملفات البايثون (مثل app.py و data.py وقوالب Jinja2 من مجلد templates وملفات الـ CSS).
 2. **التحليل:** توليد الـ AST الخاصة بنا وإجراء التحليل الدلالي (Semantic Analysis) حيث يتم إظهار كل الأخطاء النحوية للمستخدم إن وجدت، وتصدير الأخطاء الدلالية بشكل مفهوم عبر تقرير مخصص.
 3. **توليد الكود وإخراجه:** يتم دمج سياق البايثون (Context) مع شجرة قوالب Jinja لتوليد ملفات HTML نهائية وإخراجها في مجلد output/.
 4. **نسخ الملفات الثابتة:** يتم نسخ الملفات المساعدة كـ app.py و style.css كما هي إلى بيئة الخرج دون تعديلها، كما هو مطلوب في معمارية المترجم.
 5. **تصدير التقارير:** يتم إخراج الـ AST بشكل مفهوم يعبر عن شكل البرنامج الفعلي في ملفات بصيغة JSON (مثل ast_python.json و ast_jinja.json)، وكذلك يتم تصدير الأخطاء وسجل التوليد في ملفات نصية (semantic_report.txt و generation_log.txt) داخل مجلد مخصص يدعى compiler_output/.
- وبهذا يكون المترجم قادراً على تحويل المنطق المعقد للقوالب وكود البايثون إلى واجهات صالحة للعمل في المتصفح بكل كفاءة واحترافية.